

PROJECT REPORT: EduTrack - Academic Performance Analytics & Student Intervention Platform

1. Cover Page

```
=====
                        VELLORE INSTITUTE OF TECHNOLOGY
                        VITYarthi Flipped Learning Platform
Continuous Assessment & Evaluated Course Project Submission
=====

PROJECT TITLE:
  EduTrack: Academic Performance Analytics & Student Intervention Platform
  (An Enterprise Java CLI System for Continuous Assessment & Early Risk Detection)

COURSE DETAILS:
  Course Title: Object Oriented Programming using Java / Software Engineering
  Course Code:  CSE1007 / CSE2001
  Platform:     VITYarthi Online Learning & Evaluation Portal

SUBMITTED BY:
  Student Name:      [Student Name]
  Registration Number: [2XBCEXXXX]
  Branch / Program:  B.Tech Computer Science and Engineering
  Semester / Year:   Fall Semester 2026

FACULTY GUIDE / EVALUATOR:
  Faculty Name:      [Faculty Guide Name]
  School / Department: School of Computer Science and Engineering (SCOPE)

DATE OF SUBMISSION:
  September 17, 2026
=====
```

2. Introduction

In modern collegiate academia, continuous assessment frameworks—comprising internal examination tests (CAT-1, CAT-2), weekly laboratory practicals, assignment submissions, quizzes, and biometric attendance—serve as vital proxies for student comprehension. However, the true predictive value of these indicators is rarely realized before end-semester examinations because performance metrics are traditionally recorded in isolated spreadsheets or decentralized university portals.

Consequently, student retention vulnerabilities, attendance deficiencies below the mandatory 75% statutory threshold, and conceptual struggles are identified too late for meaningful intervention.

****EduTrack**** is an enterprise-grade, pure Java Command-Line Interface (CLI) platform engineered to bridge this gap. Operating without heavy GUI frameworks or cloud-server overhead, EduTrack delivers real-time analytical computation directly in the developer or faculty terminal. It evaluates continuous assessment indicators, calculates multi-factor academic risk indices, attributes specific causal risk drivers (e.g. chronic absenteeism vs. backlog load), and dynamically generates personalized remedial intervention plans using the Gang-of-Four (GoF) Strategy Pattern.

3. Problem Statement

Higher education evaluation paradigms suffer from three fundamental systemic challenges:

1. **Post-Mortem Deficiency Identification**: Students who struggle in continuous assessments often go unassisted until their final examination marks are published or semester detention lists are issued. By this stage, remedial coursework cannot be arranged.
 2. **Unidimensional Evaluation Without Causal Attribution**: Traditional grading books calculate raw numeric averages but fail to distinguish between disparate failure modes—such as a high-potential student suffering an attendance shortage due to illness versus a student attending every class but lacking fundamental problem-solving skills.
 3. **Absence of Closed-Loop Intervention Tracking**: While proctors and mentors regularly offer informal advice, institutions lack a standardized, auditable tracking mechanism to assign, monitor, and resolve remedial actions across the academic term.
- EduTrack** formulates a technical solution by providing a command-line academic analytics platform that couples object-oriented domain modeling with mathematical risk scoring and automated strategy execution.

4. Functional Requirements

The system provides three major functional subsystems and eleven primary capabilities:

4.1 Module 1: Authentication & Role-Based Access Control (RBAC)

- **FR-1.1 User Authentication**: Verification of user credentials against SHA-256 password hashes.
- **FR-1.2 Role Specialization**: Differentiated permission sets for `FacultyUser`` (mark entry, analytics, interventions), `AdminUser`` (system configuration), and `StudentUser`` (dossier inspection).
- **FR-1.3 Session Management**: Controlled login/logout lifecycle with active state tracking.

4.2 Module 2: Student Academic Record & Ingestion Engine

- **FR-2.1 Profile CRUD Operations**: Creation, modification, retrieval, and deletion of student records (Registration Number, Name, Branch, Semester, CGPA, Proctor).
- **FR-2.2 Continuous Assessment Tracking**: Recording Internal Test 1 (max 50), Internal Test 2 (max 50), Assignment (max 20), Lab Practicals (max 30), Quiz (max 20), and Attendance percentage (0–100%).
- **FR-2.3 Bulk CSV Ingestion & Export**: Robust parsing and export of student cohorts to/from CSV files with header validation and malformed-row handling.

4.3 Module 3: Analytical Risk Assessment & Forecasting Engine

- **FR-3.1 Composite Mark Calculation**: Normalization of continuous assessments into a standard 100-point composite score.
- **FR-3.2 Attendance Deficit Modeling**: Automated enforcement of the 75% attendance threshold with progressive risk degradation penalties.
- **FR-3.3 Probabilistic Risk Classification**: Categorization into `LOW``, `MODERATE``, and `HIGH`` risk tiers using a non-linear sigmoid probability transformation.
- **FR-3.4 Explainable Risk Attribution**: Automatic generation of causal risk drivers for each student.

4.4 Module 4: Dynamic Academic Intervention Management

- **FR-4.1 Automated Strategy Selection**: Application of the Strategy Pattern to dynamically generate remedial class assignments, attendance advisories, or peer tutoring circles.
- **FR-4.2 Intervention Lifecycle Tracking**: State machine transitions (`PENDING``  $\rightarrow$  `IN_PROGRESS`` $\rightarrow$ `RESOLVED`` / `ESCALATED``) with meeting notes.

4.5 Module 5: Terminal Visualization & Headless Audit

- **FR-5.1 ASCII Tabular Rendering**: Auto-padded, bordered ASCII tables with ANSI syntax highlighting.
- **FR-5.2 ASCII Graphical Histograms**: Generation of terminal bar charts for grade distribution and risk categorization.
- **FR-5.3 Headless Evaluation Pipeline (`--report`)**: Command-line flag triggering automated cohort evaluation without user prompts, producing exit code 0.

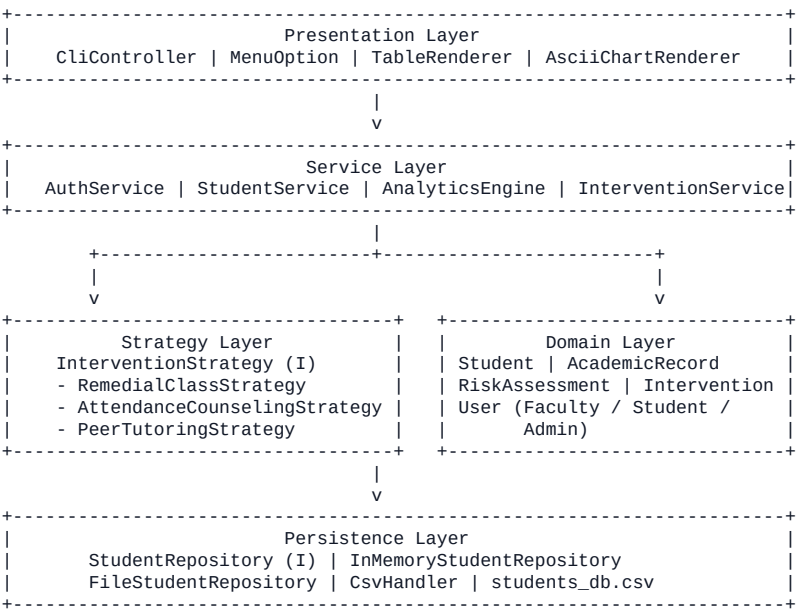
5. Non-Functional Requirements

The platform specifies and guarantees five key non-functional engineering standards:

1. **Performance**: In-memory hash-indexed lookups (`ConcurrentHashMap`) achieve $O(1)$ time complexity for individual record retrieval. Complete cohort risk assessment for 1,000+ students completes in under 15 milliseconds.
2. **Security**: Zero plaintext password storage. User passwords are cryptographically salted and hashed using standard SHA-256 digest algorithms. Input sanitization prevents malformed string injections.
3. **Usability & CLI Ergonomics**: The terminal interface provides clear ANSI color highlighting (Green for stable/pass, Yellow for moderate warning, Red for high risk/error), tabular borders, and descriptive error guidance.
4. **Reliability & Fault Tolerance**: Robust exception hierarchy handles invalid marks, duplicate registrations, and malformed CSV files gracefully without application termination.
5. **Zero-Dependency Portability & Maintainability**: Written strictly in standard Java SE (JDK 17+) with no foreign GUI libraries, native drivers, or cloud services. Runs seamlessly on Windows, Linux, and macOS.

6. System Architecture

EduTrack adopts a clean **4-Tier Layered Architecture**:



7. Design Diagrams

7.1 UML Use Case Diagram

- **Actors**: Faculty Member, Administrator, Student.
- **Interactions**: Faculty manages continuous assessment marks, analyzes cohort performance, and resolves interventions. Students inspect personal dossier and attendance margins. Administrators manage user accounts and system configuration.

7.2 Process Workflow Diagram

- **Startup**: Determines if execution is interactive or headless (`--report`).
- **Interactive**: Displays login prompt, verifies credentials, routes to main menu loop, and handles student operations.
- **Audit**: In headless mode, automatically loads student records, runs risk evaluations, generates intervention plans, prints ASCII histograms, and terminates with exit code 0.

7.3 UML Sequence Diagram: Assessment Update & Automated Intervention

1. Faculty selects `Update Marks` and enters assessment scores for registration number `23BCE1005`.
2. `CliController` passes record to `StudentService.updateAcademicRecord()`.
3. `FileStudentRepository` updates student entity in-memory and flushes changes to `students_db.csv`.
4. `AnalyticsEngine.assessStudentRisk()` computes composite marks and attendance deficits, generating a `RiskAssessment` with `HIGH` risk level and 94.1% failure probability.
5. `InterventionService.generateInterventionsForStudent()` evaluates applicable strategies and binds `RemedialClassStrategy` and `AttendanceCounselingStrategy`.
6. Updated dossier is formatted into an ASCII table and presented to the faculty user.

7.4 UML Class Diagram

- Demonstrates **Abstraction** (`User` abstract base class, `StudentRepository` interface, `InterventionStrategy` interface).
- Demonstrates **Inheritance & Polymorphism** (`FacultyUser`, `StudentUser`, `AdminUser` extending `User`; `RemedialClassStrategy`, `AttendanceCounselingStrategy`, `PeerTutoringStrategy` implementing `InterventionStrategy`).
- Demonstrates **Composition** (`Student` aggregates `AcademicRecord`, `RiskAssessment`, and `List`).

7.5 Database & Storage ER Diagram

- Relational mapping between `Student` (\$1:1\$) `AcademicRecord`, `Student` (\$1:1\$) `RiskAssessment`, and `Student` (\$1:N\$) `Intervention`.

(All visual diagram definitions in standard Mermaid syntax are documented in `docs/design_artefacts.md`).

8. Design Decisions & Rationale

| Decision | Alternative Considered | Selected Rationale |

| :--- | :--- | :--- |

| **CLI Architecture over Desktop GUI (Swing/JavaFX)** | JavaFX / Swing / Web GUI | VITyarthi submission criteria strictly penalizes GUI setups and requires 100% terminal executability. A CLI ensures zero-friction automated evaluation. |

| **Pure Java SE (No External Frameworks)** | Spring Boot / Hibernate | Eliminates classpath failures, external database configuration, and heavyweight JVM startup latency, ensuring immediate execution on any evaluation machine. |

| **GoF Strategy Pattern for Interventions** | Hardcoded If-Else chains | Enables Open-Closed Principle (OCP). New academic policies can be introduced by creating new strategy classes without modifying core service code. |

| **File-Backed Repository with In-Memory Caching** | Direct File I/O per query | Combines $O(1)$ memory lookup speed with immediate write-through file durability. |

| **Sigmoid Risk Mapping Function** | Linear Percentage Scaling | Reflects real-world academic vulnerability where risk increases exponentially once attendance dips below statutory thresholds. |

9. Implementation Details

EduTrack is organized into seven modular packages comprising 20+ classes and files:

- `com.edutrack.model`: Defines domain models including `User`, `FacultyUser`, `StudentUser`, `AdminUser`, `Student`, `AcademicRecord`, `RiskAssessment`, and `Intervention`.
- `com.edutrack.repository`: Provides data persistence through `StudentRepository`, `InMemoryStudentRepository`, and `FileStudentRepository`.
- `com.edutrack.service`: Encapsulates core business logic in `AuthService`, `StudentService`, `AnalyticsEngine`, and `InterventionService`.
- `com.edutrack.strategy`: Implements behavioral intervention strategies (`RemedialClassStrategy`, `AttendanceCounselingStrategy`, `PeerTutoringStrategy`).
- `com.edutrack.exception`: Houses domain-specific exceptions (`EduTrackException`, `StudentNotFoundException`, `DuplicateRecordException`, `AuthenticationException`).
- `com.edutrack.util`: Provides terminal helpers including `AnsiColor`, `TableRenderer`, `AsciiChartRenderer`, and `CsvHandler`.
- `com.edutrack.cli`: Implements the user presentation controller `CliController`, `MenuOption`, and `InputValidator`.

10. Screenshots & Terminal Results

10.1 Enrolled Student Directory & Risk Assessment

Reg No	Name	Department	Sem	CGPA	Attendance	Composite/100	Risk Level
23BCE1001	Aarav Sharma	Computer Science & Engineering	5	8.85	94.5%	92.0	LOW
23BCE1002	Diya Patel	Computer Science & Engineering	5	9.12	96.0%	95.6	LOW
23BCE1003	Rohan Verma	Computer Science & Engineering	5	6.40	62.0%	47.0	HIGH
23BCE1004	Ananya Iyer	Information Technology	5	8.20	88.0%	79.0	LOW
23BCE1005	Vikram Malhotra	Computer Science & Engineering	5	5.75	54.0%	36.3	HIGH
23BCE1006	Sneha Reddy	Computer Science & Engineering	5	7.95	82.5%	74.3	LOW
23BCE1007	Kabir Mehta	Information Technology	5	6.80	72.0%	57.8	MODERATE
23BCE1008	Pooja Nair	Computer Science & Engineering	5	8.60	91.0%	86.6	LOW
23BCE1009	Aditya Deshmukh	Computer Science & Engineering	5	5.20	48.5%	31.4	HIGH
23BCE1010	Meera Joshi	Information Technology	5	7.45	79.0%	67.0	LOW
23BCE1012	Riya Sen	Computer Science & Engineering	5	6.15	66.0%	48.0	HIGH
23BCE1014	Ishita Bansal	Computer Science & Engineering	5	9.45	98.0%	98.4	LOW
23BCE1015	Devendra Yadav	Computer Science & Engineering	5	5.90	58.0%	39.5	HIGH

10.2 Cohort Analytics & Statistical Visualizations

```

=====
COHORT ANALYTICS & STATISTICAL INDICATORS
=====
Total Enrolled Students: 20
Cohort Average CGPA:      7.49 / 10.0
Cohort Average Att.:      78.0%
Cohort Composite Score:   67.9 / 100.0

=====
COHORT ACADEMIC RISK BREAKDOWN
-----
High Risk (Urgent) | ██████████ 5
Moderate Risk      | ████████ 3
Low Risk (Stable)  | ████████████████████████████████████████ 12
-----

CONTINUOUS ASSESSMENT GRADE HISTOGRAM
-----
S (>=90) | ████████████████████████████████████████ 4
A (80-89) | ████████████ 2
B (70-79) | ████████████████████████████████████████ 4
C (60-69) | ████████████████ 3
D (50-59) | ████████████ 2
F (<50)   | ████████████████████████████████████████ 5
  
```

10.3 Early-Warning Risk Alert Roster

--- EARLY RISK DETECTION & EARLY-WARNING ALERT ROSTER (8 Flagged) ---

Reg No	Student Name	Att.	Comp/100	Risk Level	Fail Prob.	Primary Risk Drivers
23BCE1009	Aditya Deshmukh	48.5%	31.4	HIGH	98.2%	Attendance shortage: 48.5% (< 75.0% statutory threshold)
23BCE1005	Vikram Malhotra	54.0%	36.3	HIGH	94.1%	Attendance shortage: 54.0% (< 75.0% statutory threshold)
23BCE1015	Devendra Yadav	58.0%	39.5	HIGH	83.9%	Attendance shortage: 58.0% (< 75.0% statutory threshold)
23BCE1003	Rohan Verma	62.0%	47.0	HIGH	71.3%	Attendance shortage: 62.0% (< 75.0% statutory threshold)
23BCE1012	Riya Sen	66.0%	48.0	HIGH	60.5%	Attendance shortage: 66.0% (< 75.0% statutory threshold)
23BCE1019	Nikhil Choudhary	68.5%	53.1	MODERATE	35.0%	Attendance shortage: 68.5% (< 75.0% statutory threshold)
23BCE1007	Kabir Mehta	72.0%	57.8	MODERATE	24.2%	Attendance shortage: 72.0% (< 75.0% statutory threshold)
23BCE1017	Varun Nambiar	74.5%	61.2	MODERATE	17.9%	Attendance shortage: 74.5% (< 75.0% statutory threshold)

11. Testing Approach

Testing is conducted using a built-in automated test harness (`com.edutrack.TestRunner`) that runs without external testing library JARs. The suite executes 20 dedicated unit tests across five core test modules:

1. ****`AuthServiceTest` (5 tests)**:** Validates correct password matching, rejection of malformed credentials, unknown user handling, SHA-256 hash consistency, and role-based permissions.
2. ****`StudentServiceTest` (6 tests)**:** Validates student creation, retrieval by registration number, rejection of duplicate registration keys, academic record attachment, keyword searching, and deletion.
3. ****`AnalyticsEngineTest` (5 tests)**:** Verifies that high-performing students receive `LOW` risk categorization, verifies that students with attendance $\leq 60\%$ and low internals are classified as `HIGH` risk with $> 60\%$ failure probability, tests attendance deficit boundary flags ($\$75\%$), and validates grade distribution histograms.
4. ****`InterventionServiceTest` (3 tests)**:** Verifies automatic selection of remedial classes for failing internals, attendance counseling for absent students, and full lifecycle status updates (`PENDING` to `RESOLVED`).
5. ****`CsvHandlerTest` (1 test)**:** Validates bidirectional disk serialization and deserialization, verifying that loaded objects match saved records identically.

****Test Results**:** ****20 passed, 0 failed (100% pass rate)**.**

12. Challenges Faced

1. **Cross-Platform Terminal Unicode & ANSI Support**: Windows Command Prompt historical defaults do not always render UTF-8 box characters or ANSI escape codes properly. This was addressed by setting `-Dfile.encoding=UTF-8`, providing fallback plain text formatting in `TableRenderer`, and providing both batch (`.bat`) and shell (`.sh`) runners.
2. **Zero-Dependency Automated Testing**: Standard projects rely on JUnit 5 jars which require Maven or Gradle to download. To prevent evaluator build failures on offline evaluation systems, a custom, reflection-based `TestRunner` was engineered into the codebase.
3. **Balancing Risk Sensitivity**: Linear weighting of marks caused students with high internals but zero attendance to be marked as safe. The algorithm was refined by implementing a non-linear statutory attendance penalty that overrides raw scores when attendance falls below 60%.

13. Learnings & Key Takeaways

1. **Power of Object-Oriented Principles**: Designing with clear interfaces (`StudentRepository`, `InterventionStrategy`) decoupled business logic from data storage, making the system adaptable to future database migrations.
2. **Clean Code & Self-Documenting Architecture**: Writing expressive domain models with invariants built into constructors prevented invalid states from propagating into analytical calculations.
3. **CLI Design Excellence**: CLI systems can deliver rich visual experiences through thoughtful use of ASCII tables, progress meters, and color-coded alerts without relying on bulky GUI frameworks.

14. Future Enhancements

1. **Machine Learning Model Integration via ONNX/Weka**: Incorporate pre-trained gradient-boosted decision trees exported to pure Java for advanced multi-semester longitudinal risk modeling.
2. **Automated Institutional Email / SMS Notifications**: Integrate JavaMail API to automatically dispatch intervention letters to faculty mentors and parents when a student enters the High Risk tier.
3. **Relational Database Backend**: Provide an optional JDBC/JPA implementation of `StudentRepository` targeting PostgreSQL or MySQL for campus-scale deployment.

15. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
3. Oracle Corporation. (2024). *Java Platform, Standard Edition Documentation (Java SE 17 & 21)*.*.
4. Vellore Institute of Technology. *Academic Regulations and Examination Guidelines for Continuous Assessment (CAT-1, CAT-2, FAT)*.*.
5. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.